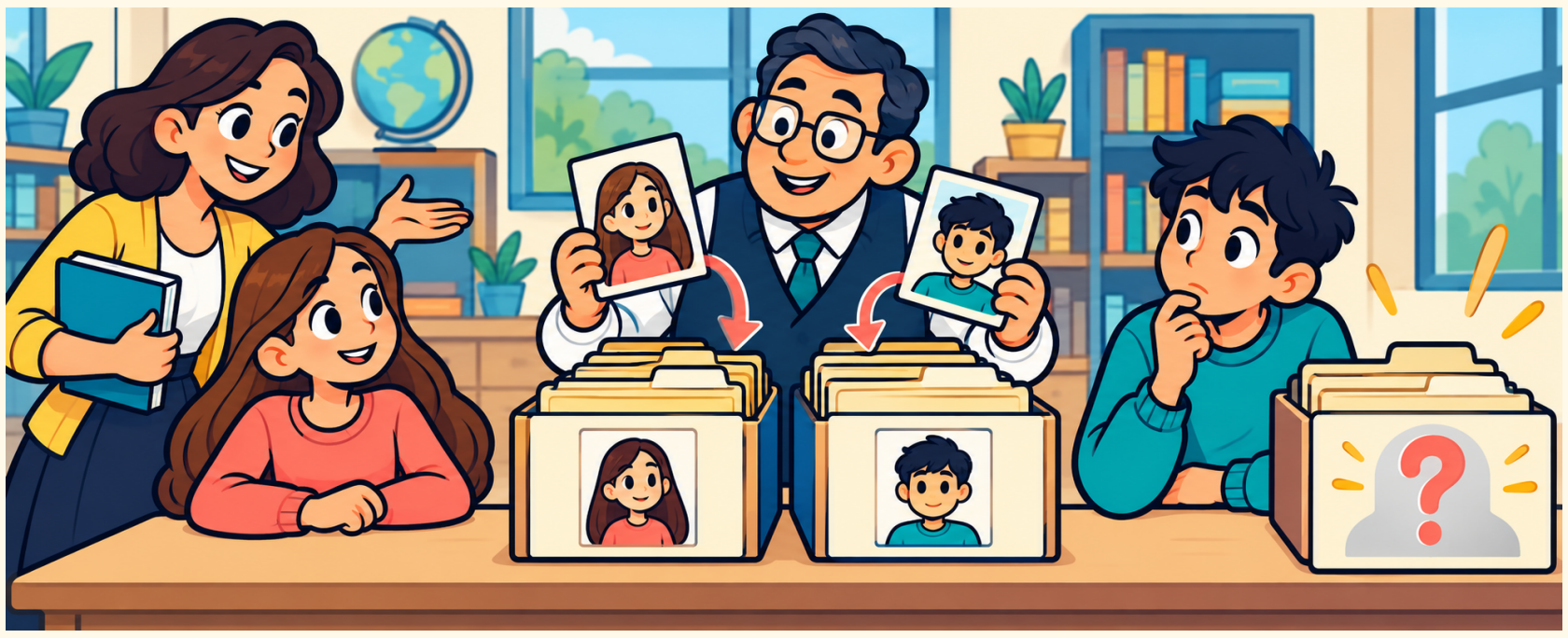
1:1 · Todo encaja. ¡El mundo es feliz!

Cada alumno tiene, como máximo, una ficha. En estos datos, ambos la tienen.



ALUMNO 1 — 0..1 FICHA

ALUMNO

id	nombre
1	Ana
2	Bruno

FICHA

alumno_id	ciudad
1	Madrid
2	Lima

```
SELECT a.nombre, f.ciudad
FROM alumno a
JOIN ficha f ON f.alumno_id = a.id;
```

LO QUE DEVUELVE LA CONSULTA

nombre	ciudad
Ana	Madrid
Bruno	Lima

Una ficha por alumno = como máximo una coincidencia

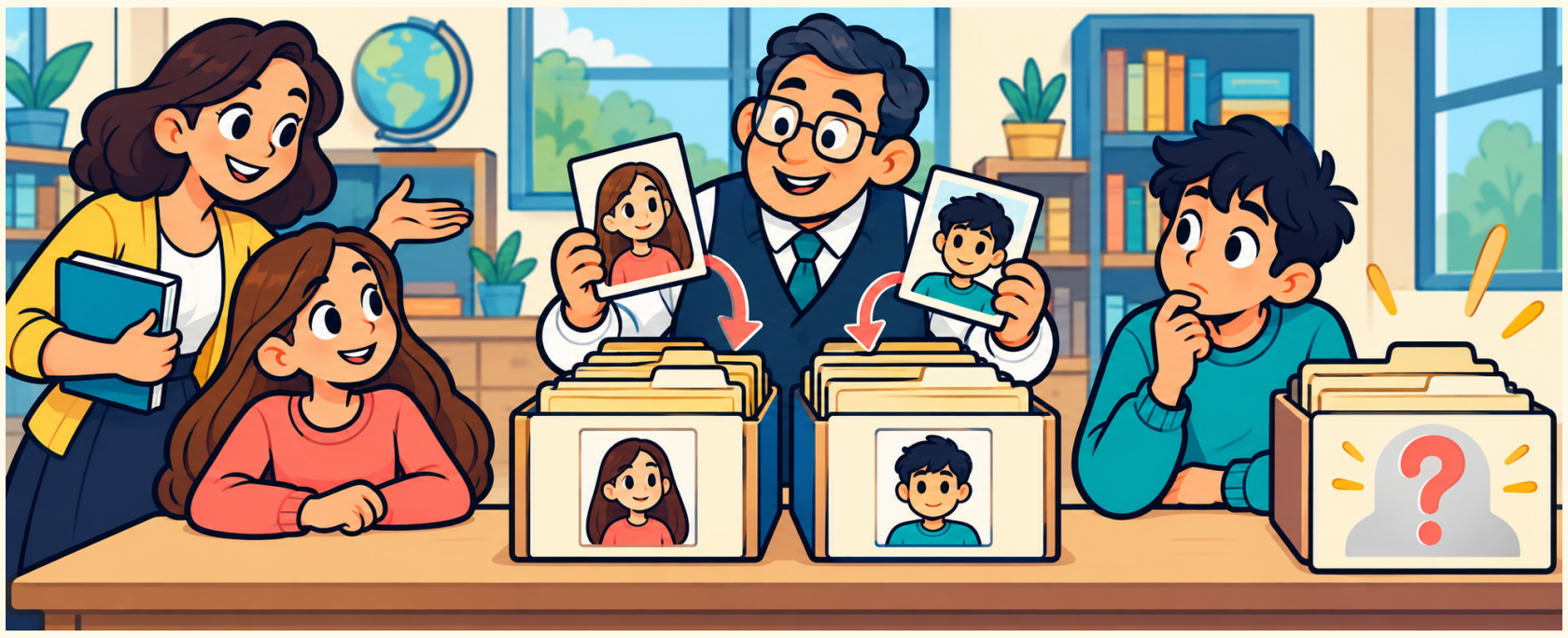
JOIN, sin más, significa INNER JOIN: muestra las parejas que coinciden. La condición ON une identificadores iguales.

Traducción: SELECT elige columnas; FROM indica dónde empezar. Las letras a y f son apodos de las tablas. **f.alumno_id = a.id** busca la ficha de ese alumno.

Regla en la base: ficha.alumno_id es PK y FK hacia alumno.id. PK evita dos fichas del mismo alumno; FK evita fichas de alumnos inexistentes.

1:1 · Falta una ficha. ¿Y Bruno?

Ahora solo existe la ficha de Ana. El registro de Bruno sigue en ALUMNO.



ALUMNO

id	nombre
1	Ana
2	Bruno

FICHA

alumno_id	ciudad
1	Madrid

```
SELECT a.nombre, f.ciudad
FROM alumno a
LEFT JOIN ficha f ON f.alumno_id = a.id;
```

CON JOIN / INNER JOIN

nombre	ciudad
Ana	Madrid

CON LEFT JOIN

nombre	ciudad
Ana	Madrid
Bruno	NULL

LEFT: «No dejes fuera a nadie de la izquierda»

ALUMNO está a la izquierda del JOIN. Bruno aparece aunque no haya ficha. NULL significa que no hay un valor disponible; no es el texto vacío ni el número cero.

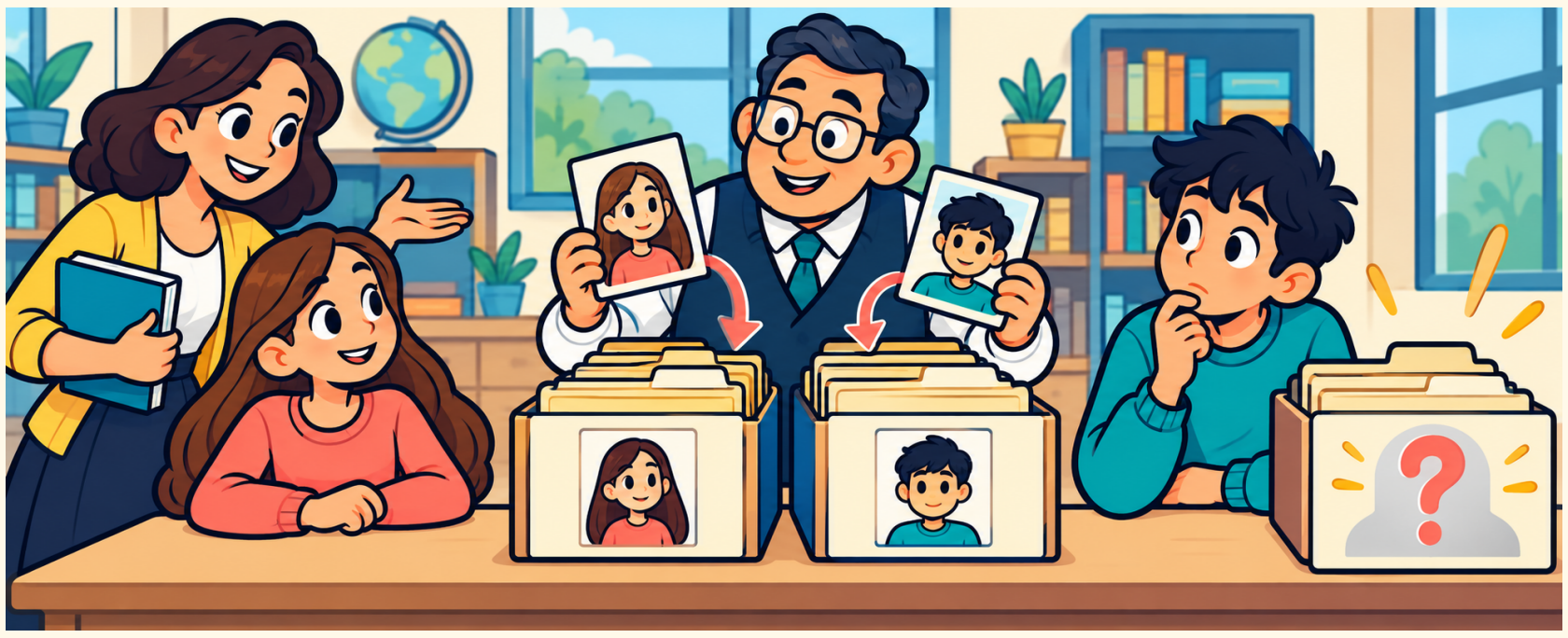
No se ha borrado nada: INNER solo excluía a Bruno del resultado de esa consulta. LEFT lo conserva en el resultado.

¿Y FULL OUTER JOIN?

Aquí da lo mismo que LEFT: no existen fichas sin alumno, porque la FK lo impide y consultamos todos los alumnos. En la próxima lámina veremos cuándo FULL sí añade algo.

FULL · Nadie queda fuera de las dos listas

Un ejemplo de conciliación: las listas de dos actividades, con una fila por alumno.



LISTA_A · MAÑANA

id	nombre
1	Ana
2	Bruno

LISTA_B · TARDE

id	nombre
1	Ana
3	Carla

```
SELECT a.nombre AS manana, b.nombre AS tarde
FROM lista_a a
FULL OUTER JOIN lista_b b ON b.id = a.id;
```

LO QUE DEVUELVE LA CONSULTA

manana	tarde
Ana	Ana
Bruno	NULL
NULL	Carla

INNER JOIN

Quién aparece
Ana

LEFT JOIN desde A

Quién aparece
Ana
Bruno

FULL OUTER JOIN

Quién aparece
Ana
Bruno
Carla

FULL conserva lo exclusivo de ambos lados

Ana coincide. Bruno está solo por la mañana. Carla está solo por la tarde. Son dos listas independientes; no estamos creando una ficha que incumpla una clave foránea.

Recuerda: LEFT y FULL deciden qué filas sin coincidencia sobreviven. Ninguno obliga a que exista una sola coincidencia.

1:N · Ana entrega dos trabajos

Ahora una alumna puede tener varias entregas. El JOIN muestra una fila por coincidencia.



ALUMNO 1 — 0..N **ENTREGA**

ALUMNO

id	nombre
1	Ana
2	Bruno

ENTREGA

id	alumno_id	puntos
101	1	10
102	1	20

```
SELECT a.nombre, e.id AS entrega, e.puntos
FROM alumno a
LEFT JOIN entrega e ON e.alumno_id = a.id;
```

LO QUE DEVUELVE LA CONSULTA

nombre	entrega	puntos
Ana	101	10
Ana	102	20
Bruno	NULL	NULL

Ana aparece dos veces, pero no está duplicada en ALUMNO

Son dos filas diferentes del resultado: una por cada entrega. Cada una aporta información distinta. Es lo correcto si quieres ver el detalle.

INNER: 2 filas de Ana. **LEFT:** esas 2 + Bruno. **FULL:** igual que LEFT en estos datos, porque no hay entregas sin alumno. **Cambiar el JOIN no resume las entregas.**

1:N · Si quieres una fila, resume primero

GROUP BY prepara una ficha resumen por alumno; después LEFT JOIN la incorpora.



ENTREGA · ANTES

alumno_id	puntos
1	10
1	20

RESUMEN · DESPUÉS

alumno_id	total
1	30

```
WITH resumen AS (
  SELECT alumno_id, SUM(puntos) AS total
  FROM entrega
  GROUP BY alumno_id
)
SELECT a.nombre, COALESCE(r.total, 0) AS total
FROM alumno a
LEFT JOIN resumen r ON r.alumno_id = a.id;
```

LO QUE DEVUELVE LA CONSULTA

nombre	total
Ana	30
Bruno	0

Tres palabras, tres tareas

GROUP BY: reúne las filas de cada alumno.
SUM: suma sus puntos.
WITH: da un nombre al resultado calculado para usarlo después.

COALESCE(r.total, 0) muestra 0 cuando no hay resumen. Aquí tiene sentido: no entregar trabajos significa sumar cero puntos.

No has cambiado la relación original

ALUMNO sigue teniendo 1:N entregas. Solo el resumen calculado aporta 0..1 filas por alumno. Así LEFT JOIN devuelve exactamente una fila por alumno.

N:N · La matrícula es el puente

Ana puede estudiar varios cursos. Un mismo curso puede tener varios alumnos.



ALUMNO

id	nombre
1	Ana
2	Bruno
3	Carla

MATRICULA

alumno_id	curso_id
1	10
1	20
2	10

CURSO

id	titulo
10	SQL
20	Python
30	Linux

Así se transforma el modelo entidad-relación



Una sola tabla puente, dos relaciones 1:N. 0..N significa ninguna, una o muchas matrículas.

Cada fila de MATRICULA cuenta una historia

(1,10): Ana estudia SQL.
(1,20): Ana estudia Python.
(2,10): Bruno estudia SQL.

Dos referencias: alumno_id apunta a ALUMNO; curso_id apunta a CURSO. Ambas son claves foráneas, FK.

La clave primaria es la pareja

PRIMARY KEY (alumno_id, curso_id) impide matricular dos veces al mismo alumno en el mismo curso. Cada identificador por separado sí puede repetirse.

No unas alumno.id = curso.id: son identificadores de cosas distintas. La tabla puente indica qué parejas existen. Carla aún no está matriculada; Linux aún no tiene alumnos.

N:N · Recorre el puente con dos JOIN

Mismos datos de la lámina anterior. Primero alumno → matrícula; después matrícula → curso.



```
SELECT a.nombre, c.titulo
FROM alumno a
JOIN matricula m ON m.alumno_id = a.id
JOIN curso c ON c.id = m.curso_id;
```

CON LOS DOS JOIN / INNER JOIN

nombre	titulo
Ana	SQL
Ana	Python
Bruno	SQL

LOS DOS LEFT JOIN

nombre	titulo
Ana	SQL
Ana	Python
Bruno	SQL
Carla	NULL

LOS DOS FULL OUTER JOIN

nombre	titulo
Ana	SQL
Ana	Python
Bruno	SQL
Carla	NULL
NULL	Linux

Prueba cambiando ambos JOIN en la consulta

INNER: solo matrículas con sus alumnos y cursos.
LEFT: conserva también a Carla, sin matrícula.
FULL: conserva además Linux, sin alumnos.

Ojo con mezclar: LEFT hasta la matrícula y luego INNER hasta el curso puede volver a excluir a Carla. Sigue el camino completo. Los ejemplos no fijan el orden de salida: usa ORDER BY si lo necesitas.

N:N · ¿Cuántos cursos tiene cada alumno?

Una fila por alumno: agrupa el puente y luego incorpora el resumen.



```
WITH resumen AS (
  SELECT alumno_id, COUNT(*) AS cursos
  FROM matricula
  GROUP BY alumno_id
)
SELECT a.nombre, COALESCE(r.cursos, 0) AS cursos
FROM alumno a
LEFT JOIN resumen r ON r.alumno_id = a.id;
```

LO QUE DEVUELVE LA CONSULTA

nombre	cursos
Ana	2
Bruno	1
Carla	0

COUNT(*) cuenta matrículas dentro del puente

Como la pareja alumno-curso es única, aquí una matrícula equivale a un curso de ese alumno. No hace falta unir CURSO si solo quieres contar.

¿Por qué no usar COUNT(*) después de un LEFT JOIN?

Carla tendría una fila de relleno y podrías contar 1. Si agrupas después del LEFT, cuenta una columna no nula de la matrícula: COUNT(m.curso_id). Así Carla da 0.

La escalera completa

1:1: como máximo una coincidencia.
1:N: una fila por cada coincidencia.
N:N: una fila por cada asociación del puente.
GROUP BY: cambia el nivel de detalle del resultado.
INNER / LEFT / FULL: deciden quién se queda fuera o dentro.